

Introduction

If you've built more than a couple of n8n workflows, you've probably hit this wall: everything works perfectly against a handful of test rows, then you point the same workflow at real production data and it dies partway through. Not with a helpful error message pointing at your logic — just a blunt `429 Too Many Requests` from whatever API you're calling, somewhere around item 200 of 10,000.

This isn't really an n8n problem. It's a systems design problem that shows up constantly once you move from toy datasets to production scale, and it maps almost exactly onto something interviewers love to probe: how do you handle rate-limited, high-volume operations without falling over? Whether you're calling a REST API from a Python script, hitting a third-party service from a backend job, or wiring up a no-code workflow, the underlying mechanism — and the fix — is the same. Understanding it here will save you from re-learning it the hard way in a codebase where the failure is buried three services deep.

Problem Overview

Picture a workflow that reads rows from a spreadsheet or database and, for each row, fires off a request to an external API — enriching a lead, fetching a price, whatever the use case is. With 10 rows, this is instant and invisible. With 10,000 rows, the loop node inside the workflow doesn't process them one at a time with any natural pacing — it fires all of them essentially simultaneously.

From the API's point of view, it just got hit with 10,000 requests in the same instant. Every API that matters protects itself from exactly this: it enforces a rate limit, and once you cross it, it starts rejecting requests with a `429` status code. Some providers go further and temporarily suspend the API key entirely if you keep hammering past the limit after being told to back off.

The fix isn't a queueing system, a message broker, or some heavyweight infrastructure. It's much simpler: don't send everything at once. Split the list into small batches, process one batch, pause, then move to the next. The API sees a steady trickle instead of a flood, and it never has a reason to reject you.

Example

Say you have 10,000 rows and the API allows 100 requests per second before it starts throttling.

- **Unbatched:** all 10,000 requests fire within the same second or two → the API's rate limiter kicks in almost immediately → you start seeing 429 s a couple hundred rows in.
- **Batched at 100, with a 1-second pause between batches:** you send 100 requests, wait a second, send the next 100, and so on. You finish in roughly 100 seconds, and the API never sees more than 100 requests in any given second.

Same total work, same total number of requests — the only difference is pacing. That's the entire fix.

Intuition

The instinct when something "breaks at scale" is to assume the logic is wrong. But nothing about the per-row logic changed between 10 rows and 10,000 rows — the HTTP call, the field mapping, the response handling, all identical. What changed is volume and timing.

An experienced engineer's first question when a system fails only at scale is: *what resource am I contending for, and does it have a limit?* Here, the resource is the API's rate limit, and the limit is per second (or per minute), not a hard cap on total requests. That reframes the problem immediately — you don't need to reduce how much data you send, you need to control the *rate* at which you send it. That's the difference between a batching problem and a data problem, and it's why "just send fewer rows" is the wrong fix while "send the same rows, more slowly" is the right one.

Once you frame it as a rate problem, the solution follows naturally: chunk the work into batches, and insert a deliberate pause between chunks so the sending rate stays under whatever ceiling the API enforces.

Brute Force Solution

Idea: Do nothing — let the loop node process every item as fast as it can.

Algorithm: Iterate over all 10,000 rows, fire a request for each, immediately, with no pacing.

Advantages: Zero extra setup. Works fine for small datasets, quick prototypes, or APIs with no meaningful rate limit.

Disadvantages: Falls over the moment the dataset is large enough or the API strict enough. Failures are non-deterministic in the sense that you don't know exactly how many rows got through before the rejections started, which makes recovery painful.

Time complexity: $O(n)$ requests, sent effectively all at once — the wall-clock time is dominated by how fast the API starts rejecting you, not by the actual work.

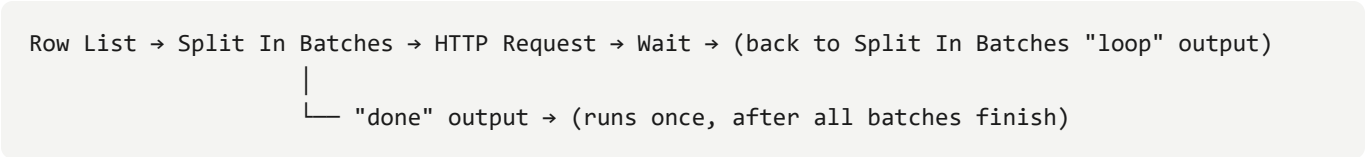
Space complexity: $O(1)$ beyond holding the row list, since nothing is being tracked between iterations.

Optimal Solution

The fix has two parts working together:

- 1. **Split In Batches** — caps how many items go out per pass (volume control).
- 2. **A Wait node inside the loop** — caps how often a pass happens (speed control).

Here's the flow:



The Split In Batches node has two outputs, and getting them backwards is the single most common mistake:

Output	Purpose	Wire it to
loop	Fires once per batch, with the current batch's items	Your processing nodes, which then feed back into Split In Batches
done	Fires exactly once, after every batch has been processed	Whatever should run <i>after</i> the entire run completes

If you wire `loop` back into itself without ever routing through `done` , you get one of two failure modes: an infinite loop that never terminates, or a workflow that only ever processes the first batch and then looks finished when it isn't.

The row list connects into `Split In Batches` first — not directly into the HTTP Request node. That's the piece people skip when converting an existing per-row workflow. Set **Batch Size** (say, 100) on the node; the HTTP Request node underneath only ever sees the current batch, pulled in through the item/body expression. Add a **Wait** node inside the loop, after the HTTP Request, with a short pause (e.g., 1 second) before looping back — this is what actually enforces the API's requests-per-second limit, not the batch size alone.

Python Code

n8n's Split In Batches node is a visual equivalent of this pattern. Here's the same logic expressed in Python, since the concept transfers directly to any script or backend job that has to call a rate-limited API over a large dataset:

```

import time
from itertools import islice

def chunked(iterable, size):
    """Yield successive chunks of `size` items from `iterable`."""
    it = iter(iterable)
    while chunk := list(islice(it, size)):
        yield chunk

def process_rows(rows, call_api, batch_size=100, pause_seconds=1.0):
    """
    Process `rows` in batches, pausing between batches to respect
    an API's rate limit. Returns the list of results in order.
    """
    results = []

    for batch_number, batch in enumerate(chunked(rows, batch_size), start=1):
        for row in batch:
            results.append(call_api(row))

        # Skip the pause after the final batch – nothing follows it.
        if batch_number * batch_size < len(rows):
            time.sleep(pause_seconds)

    return results

```

Code Walkthrough

- `chunked()` is a generator that slices an iterable into fixed-size lists using `itertools.islice`, without loading batching logic into the caller. This mirrors what `Split In Batches` does internally — it hands you one batch at a time and remembers where it left off.
- The walrus operator (`chunk := list(islice(it, size))`) both assigns and tests the chunk in one line, stopping the loop cleanly once the iterator is exhausted.
- `process_rows()` is the equivalent of the whole n8n canvas: the outer `for` loop is `Split In Batches`, the inner `for` loop plus `call_api(row)` is the `HTTP Request` node, and `time.sleep(pause_seconds)` is the `Wait` node.
- The `if batch_number * batch_size < len(rows)` guard skips the pause after the last batch — equivalent to the `done` output firing without another `loop` pass.
- `call_api` is injected as a parameter rather than hardcoded, so the same batching logic works regardless of what the actual request does — same separation of concerns as n8n keeping `Split In Batches` and `HTTP Request` as distinct nodes.

Dry Run

Take 250 rows, `batch_size=100`, `pause_seconds=1.0`.

Batch Rows processed Pause after?

1	rows 1–100	yes, 1s
2	rows 101–200	yes, 1s
3	rows 201–250	no (last batch)

Total requests: 250, same as sending everything at once. Total added wall-clock time: 2 seconds, in exchange for never exceeding 100 requests in any given window. That trade — a small amount of added latency for guaranteed completion — is the whole point.

Complexity Analysis

Time: $O(n)$ requests total, same as the brute-force approach — batching doesn't change the amount of work, only its distribution over time. Wall-clock time becomes $O(n / \text{batch_size})$ pauses of `pause_seconds` each, which is a deliberate, bounded cost you control directly.

Space: $O(\text{batch_size})$ held in memory at any one time if you're streaming rows, versus $O(n)$ if you load the entire dataset upfront — either way, it's no worse than the brute-force version, and batching doesn't add meaningful memory overhead.

These bounds are correct because batching is purely a scheduling change: no additional data is created or retained, and no request count changes — only the arrival rate seen by the API.

Alternative Solutions

Token bucket / leaky bucket rate limiter: instead of fixed batches with fixed pauses, track a request budget that refills over time and only send when budget is available. More precise, especially for APIs with bursty limits, but overkill for most n8n workflows where batching plus a wait accomplishes the same thing with two nodes instead of custom logic.

Exponential backoff on 429: retry failed requests with increasing delays instead of preventing them upfront. This is a reasonable complement (handle the rare 429 that slips through) but a poor primary strategy, since you're still hammering the API and reacting to failures rather than avoiding them.

Queue-based architecture (e.g., a message queue with worker consumers): the right choice at genuinely high scale or when multiple producers/consumers are involved, but unnecessary complexity

for a single workflow hitting a single API.

Edge Cases

- **Empty row list:** the loop should exit immediately with zero batches — verify your batching logic doesn't throw on an empty input.
- **Row count exactly divisible by batch size:** make sure the last batch doesn't get a redundant pause after it, and doesn't get skipped entirely.
- **Row count smaller than batch size:** everything goes out in a single batch — this should still work without special-casing.
- **Batch size of 1:** valid but effectively removes the benefit of batching for volume, while still respecting timing if a wait is present.
- **API limit changes mid-run** (e.g., you get throttled despite batching): indicates your batch size or wait time is still too aggressive for that provider's actual limits.

Common Mistakes

1. Wiring the `loop` output back into itself while never routing anything through `done`, causing an infinite loop or a run that silently only processes the first batch.
2. Connecting the row list directly into the processing node instead of routing it through `Split In Batches` first.
3. Setting a batch size without adding a `Wait` node, which controls volume per pass but not requests per second — some APIs care about both.
4. Copying a batch size and wait time from a tutorial (e.g., "100 and 1 second") without checking the actual documented rate limit for the API being called.
5. Assuming a failed run has no state — in a properly batched workflow, you know exactly which batch and which items were in flight, so don't rebuild recovery logic that ignores that information.
6. Testing only against a small sample size (10 rows) and assuming that validates production-scale (10,000 rows) behavior — batching problems only appear at volume.

Interview Questions

- How would you handle rate-limited API calls when processing a large dataset?
- What's the difference between a rate limit that's requests-per-second versus requests-per-minute, and how would that change your batching strategy?

- How would you design a system to resume from a partial failure instead of restarting from the beginning?
- What's the tradeoff between batch size and total run time?
- How would you detect and handle a 429 response gracefully rather than letting the whole job fail?

Similar Problems

- **LeetCode 239 (Sliding Window Maximum)** — same mental model of processing data in bounded windows rather than all at once.
- **LeetCode 622 (Design Circular Queue)** — reinforces the idea of managing fixed-capacity chunks of data with controlled entry and exit.
- **Rate limiter design problems** (common in system design interviews) — directly maps to the token bucket / leaky bucket alternatives discussed above.
- **Pagination-handling problems** in API client design — the "fetch in chunks, track a cursor" pattern is structurally identical to Split In Batches remembering where it left off.

Key Takeaways

- A workflow that crashes only at production scale usually isn't a logic bug — it's a rate or resource limit you weren't respecting.
- Batching controls *volume* per request; a deliberate pause controls *speed*. You often need both.
- Get the loop / done wiring right, or you'll get an infinite loop or a workflow that silently under-processes.
- Always check the actual documented rate limit for the API you're calling instead of copying default numbers.
- This pattern — chunk, pace, repeat — applies far beyond n8n, to any script or service calling a rate-limited API at scale.

Watch the Video

For the full walkthrough, including watching the batches execute live and seeing the exact node wiring, check out the video here: {LINK}

About the Series

This lesson is part of *Fun with Learning Technology*, a series where workflows and code get pushed until they break in realistic, production-shaped ways — then fixed properly, with the actual mechanism explained rather than a patch that just hides the symptom.

Call To Action

If this saved you from a future 429 headache, give the video a like on YouTube and subscribe for the rest of the series. For deeper write-ups like this one delivered straight to your inbox, subscribe to the Substack. Drop a comment with the API you're batching against — I read every one — and share this with anyone on your team still debugging a workflow that "randomly" stops at row 200.

Quick Review

n8n: 10,000 rows fail around row 200 during item processing—what pattern fixes it?

Split In Batches: process items in chunks with a loop-back connection instead of firing all rows at once.

Why did the workflow actually break at row 200?

Downstream API rate-limited the flood of simultaneous requests; no throttling meant later calls got rejected.

Split In Batches vs a plain Loop Over Items node—when to prefer batching?

Use Split In Batches when you need chunk size control and rate/delay between chunks, not just per-item iteration.

How do you control speed, not just batch size, in the loop?

Add a Wait/delay node inside the loop body so each batch pauses before the next iteration fires.

Trickiest bug when wiring the Split In Batches loop?

Forgetting to connect the loop output back to the Split In Batches input—workflow processes only the first batch.

Interview follow-up: how would you make a failure traceable in a batched run?

Log batch index/item ID per iteration so a failure points to the exact batch and row, not the whole run.

n8n Lesson 19: Working with arrays and loops (Split In Batches) (processing large item lists in chunks, avoiding rate limits) — free Python cheat sheet